

Instituto Tecnológico de Costa Rica

Proyecto de Ingeniería de Software

Manual Técnico

Jose Pablo Granados Siles, 2022028503

Tomás Granados Preciado, 2021579524

Stiven Segura Guzmán, 2021067255

I Semestre, 2025

Índice

Índice.....	2
Arquitectura del Sistema.....	3
Visión General.....	3
Arquitectura Monorepo.....	3
Comunicación entre Módulos.....	3
Backend - Servidor API.....	4
Tecnologías y Dependencias.....	4
Estructura de Directorios.....	4
Configuración del Servidor.....	5
Base de Datos.....	6
Autenticación y Autorización.....	6
API RESTful.....	7
Middleware.....	8
Manejo de Archivos.....	8
Frontend - Aplicación Web.....	8
Tecnologías y Dependencias.....	8
Arquitectura Next.js.....	9
Estructura de Rutas.....	10
Gestión de Estado.....	10
Componentes UI.....	11
Configuración de Build.....	11
Shared - Módulo Compartido.....	12
Propósito y Función.....	12
Estructura del Módulo.....	12
Tipos y Entidades.....	12
Validaciones con Zod.....	13
Infraestructura y Despliegue.....	14
Configuración de Desarrollo.....	14
Variables de Entorno.....	15
Scripts de Build.....	15
Consideraciones de Producción.....	16
Seguridad.....	16
Autenticación JWT.....	16
Validación de Datos.....	17
CORS y Headers.....	17
Manejo de Archivos.....	17
Performance y Optimización.....	18
Optimizaciones del Frontend.....	18
Optimizaciones del Backend.....	18

Caching.....	19
Monitoreo y Logging.....	19
Sistema de Logs.....	19
Manejo de Errores.....	19
Métricas.....	20
Testing.....	20
Estrategia de Testing.....	20
Testing del Backend.....	20
Testing del Frontend.....	21
Documentación de API.....	21
Endpoints Principales.....	21
Esquemas de Datos.....	22
Códigos de Respuesta.....	23

Arquitectura del Sistema

Visión General

Colori está construido como una aplicación web full-stack moderna utilizando una arquitectura de microservicios monorepo. El sistema se compone de tres módulos principales:

- **Frontend:** Aplicación web construida con Next.js 15 y React 19
- **Backend:** API RESTful construida con Node.js, Express y TypeScript
- **Shared:** Módulo compartido con tipos, validaciones y utilidades comunes

Arquitectura Monorepo

```

PROYECTO/
├── frontend/      # Aplicación Next.js
├── backend/       # API Express.js
├── shared/        # Tipos y utilidades compartidas
└── docs/         # Documentación
  
```

Beneficios de la Arquitectura Monorepo:

- **Consistencia de tipos:** Tipos compartidos entre frontend y backend
- **Reutilización de código:** Validaciones y utilidades comunes
- **Desarrollo sincronizado:** Cambios coordinados entre módulos
- **Gestión simplificada:** Un solo repositorio para todo el proyecto

Comunicación entre Módulos

- **Frontend ↔ Backend:** Comunicación HTTP/HTTPS mediante API RESTful
- **Shared → Frontend:** Importación directa de tipos y validaciones
- **Shared → Backend:** Importación directa de modelos y esquemas

- **Tiempo Real:** WebSockets para actualizaciones en tiempo real (cocina)

Backend - Servidor API

Tecnologías y Dependencias

Core Technologies:

- **Node.js:** Entorno de ejecución JavaScript v20+
- **Express.js 4.19:** Framework web minimalista
- **TypeScript 5.4:** Lenguaje tipado estáticamente
- **MongoDB:** Base de datos NoSQL
- **Mongoose 8.3:** ODM para MongoDB

Dependencias Principales:

```
{
  "express": "^4.19.2",
  "mongoose": "^8.3.1",
  "jsonwebtoken": "^9.0.2",
  "bcryptjs": "^2.4.3",
  "express-validator": "^7.0.1",
  "multer": "^2.0.1",
  "cors": "^2.8.5",
  "zod": "^3.24.2"
}
```

Herramientas de Desarrollo:

- **tsx:** Ejecutor TypeScript para desarrollo
- **ts-node:** Transpilador TypeScript
- **morgan:** Logger de requests HTTP
- **dotenv:** Gestión de variables de entorno

Estructura de Directorios

```
backend/src/
├── config/                # Configuraciones del sistema
│   ├── db.config.ts      # Configuración de MongoDB
│   ├── env.config.ts     # Variables de entorno
│   ├── logger.config.ts  # Sistema de logging
│   └── index.ts          # Exportaciones de configuración
├── controllers/          # Controladores de rutas
│   ├── user/             # Controladores de usuarios
│   ├── product/          # Controladores de productos
│   ├── order/            # Controladores de órdenes
│   └── promotion/        # Controladores de promociones
├── middlewares/          # Middleware personalizado
└── auth.middleware.ts     # Autenticación JWT
```

```

├── validation.middleware.ts # Validación de datos
├── error.middleware.ts     # Manejo de errores
├── models/                 # Modelos de Mongoose
│   ├── user/              # Modelo de usuario
│   ├── product/           # Modelo de producto
│   ├── order/             # Modelo de orden
│   └── promotion/         # Modelo de promoción
├── routes/                 # Definición de rutas
│   ├── user/              # Rutas de usuarios
│   ├── product/           # Rutas de productos
│   ├── order/             # Rutas de órdenes
│   ├── promotion/         # Rutas de promociones
│   └── index.ts           # Agregador de rutas
├── services/              # Lógica de negocio
│   ├── user/              # Servicios de usuario
│   ├── product/           # Servicios de producto
│   ├── order/             # Servicios de orden
│   └── promotion/         # Servicios de promoción
├── types/                 # Definiciones de tipos TypeScript
└── server.ts              # Punto de entrada de la aplicación

```

Configuración del Servidor

Punto de Entrada (**server.ts**):

```

/**
 * Configuración principal del servidor Express
 * - Configuración de CORS
 * - Middleware de parsing y logging
 * - Rutas de API
 * - Manejo de errores
 * - Conexión a base de datos
 */

```

Configuración CORS:

```

const corsOptions = {
  origin: [
    "http://localhost:3000", // Frontend development
    "http://127.0.0.1:3000" // Alternative localhost
  ],
  credentials: true,
  methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
  allowedHeaders: [
    "Content-Type",
    "Authorization",
    "X-Requested-With",
    "Accept",
    "Origin"
  ]
};

```

Middleware Stack:

1. **CORS:** Configuración de políticas de origen cruzado
2. **express.json():** Parsing de JSON en requests
3. **morgan:** Logging de requests HTTP
4. **requestLogger:** Logger personalizado
5. **Static Files:** Servir archivos desde `/uploads`
6. **API Routes:** Rutas de la API bajo `/api`
7. **Error Handler:** Middleware de manejo de errores

Base de Datos

MongoDB con Mongoose:

- **Conexión:** Configurada en `db.config.ts`
- **Modelos:** Definidos con esquemas Mongoose
- **Validaciones:** Integradas en los esquemas
- **Índices:** Optimizados para consultas frecuentes

Configuración de Conexión:

```
/**
 * Configuración de MongoDB
 * - Connection string desde variables de entorno
 * - Opciones de conexión optimizadas
 * - Manejo de eventos de conexión
 * - Reconexión automática
 */
```

Esquemas Principales:

- **User:** Gestión de usuarios y roles
- **Product:** Productos del menú
- **Category:** Categorías de productos
- **Order:** Órdenes de clientes
- **Promotion:** Promociones y descuentos

Autenticación y Autorización

JWT (JSON Web Tokens):

- **Generación:** Al iniciar sesión exitosamente
- **Validación:** Middleware de autenticación
- **Expiración:** Tokens con tiempo de vida limitado

- **Refresh:** Renovación automática de tokens

Roles y Permisos:

```
enum UserRole {  
    ADMIN = "ADMIN",  
    MANAGER = "MANAGER",  
    CHEF = "CHEF",  
    SERVER = "SERVER",  
    CASHIER = "CASHIER"  
}
```

Middleware de Autorización:

- Verificación de token JWT
- Validación de roles por endpoint
- Manejo de permisos granulares

API RESTful

Estructura de Endpoints:

/api/auth/*	# Autenticación
/api/users/*	# Gestión de usuarios
/api/products/*	# Gestión de productos
/api/categories/*	# Gestión de categorías
/api/orders/*	# Gestión de órdenes
/api/promotions/*	# Gestión de promociones
/api/reports/*	# Reportes y estadísticas
/api/uploads/*	# Subida de archivos

Convenciones de API:

- **GET:** Obtener recursos
- **POST:** Crear nuevos recursos
- **PUT:** Actualizar recursos completos
- **PATCH:** Actualizar recursos parcialmente
- **DELETE:** Eliminar recursos

Formato de Respuestas:

```
interface ApiResponse<T> {  
    success: boolean;  
    data?: T;  
    message?: string;  
    error?: string;  
    pagination?: {  
        page: number;  
        limit: number;  
        total: number;  
    };  
}
```

```
    pages: number;
  };
}
```

Middleware

Middleware de Autenticación:

```
/**
 * Verifica la validez del token JWT
 * Extrae información del usuario
 * Agrega usuario al objeto request
 */
```

Middleware de Validación:

```
/**
 * Valida datos de entrada usando Zod
 * Sanitiza datos
 * Retorna errores de validación estructurados
 */
```

Middleware de Manejo de Errores:

```
/**
 * Captura errores no manejados
 * Formatea respuestas de error
 * Registra errores en logs
 * Oculta detalles sensibles en producción
 */
```

Manejo de Archivos

Multer Configuration:

- **Storage:** Almacenamiento local en `/uploads`
- **File Types:** JPG, PNG, WebP
- **Size Limits:** Máximo 5MB por archivo
- **Naming:** Nombres únicos con timestamp

Procesamiento de Imágenes:

- **Redimensionamiento:** Automático para optimización web
- **Compresión:** Reducción de tamaño sin pérdida de calidad
- **Formatos:** Conversión a WebP para mejor performance

Frontend - Aplicación Web

Tecnologías y Dependencias

Core Technologies:

- **Next.js 15.2:** Framework React con SSR/SSG

- **React 19:** Biblioteca de UI con nuevas características
- **TypeScript 5:** Tipado estático
- **Tailwind CSS 4:** Framework de estilos utility-first

UI Components:

- **Radix UI:** Componentes accesibles y sin estilos
- **Lucide React:** Iconografía moderna
- **Sonner:** Sistema de notificaciones toast
- **React Hook Form:** Gestión de formularios

Dependencias Principales:

```
{
  "next": "15.2.4",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "@radix-ui/react-*": "^1.x.x",
  "tailwindcss": "^4",
  "axios": "^1.9.0",
  "react-hook-form": "^7.57.0"
}
```

Arquitectura Next.js

App Router (Next.js 13+):

- **File-based Routing:** Rutas basadas en estructura de archivos
- **Server Components:** Renderizado del lado del servidor por defecto
- **Client Components:** Componentes interactivos marcados con 'use client'
- **Layouts:** Layouts anidados y compartidos

Configuración Next.js:

```
const nextConfig: NextConfig = {
  images: {
    domains: ["localhost"],
    remotePatterns: [
      {
        protocol: "http",
        hostname: "localhost",
        port: "3001",
        pathname: "/uploads/**"
      }
    ]
  },
  transpilePackages: ["colori-platform-shared"],
  experimental: {
```

```
    esmExternals: false
  }
};
```

Estructura de Rutas

```
frontend/app/
├── (auth)/                # Grupo de rutas de autenticación
│   ├── login/            # Página de inicio de sesión
│   └── layout.tsx        # Layout para auth
├── admin/                # Panel de administración
│   ├── dashboard/       # Dashboard principal
│   ├── users/           # Gestión de usuarios
│   ├── products/        # Gestión de productos
│   ├── promotions/      # Gestión de promociones
│   ├── reports/         # Reportes y estadísticas
│   └── layout.tsx       # Layout de admin
├── client/              # Interfaz de cliente
│   ├── menu/            # Exploración del menú
│   ├── cart/            # Carrito de compras
│   ├── checkout/        # Proceso de pago
│   └── layout.tsx       # Layout de cliente
├── kitchen/             # Panel de cocina
│   ├── orders/          # Cola de pedidos
│   └── layout.tsx       # Layout de cocina
├── globals.css          # Estilos globales
├── layout.tsx           # Layout raíz
└── page.tsx            # Página de inicio
```

Características de Routing:

- **Nested Layouts:** Layouts anidados para diferentes secciones
- **Route Groups:** Agrupación lógica con **(nombre)**
- **Dynamic Routes:** Rutas dinámicas con **[param]**
- **Loading States:** Estados de carga automáticos
- **Error Boundaries:** Manejo de errores por ruta

Gestión de Estado

Estrategias de Estado:

1. **Server State:** React Query/SWR para datos del servidor
2. **Client State:** useState y useReducer para estado local
3. **Form State:** React Hook Form para formularios
4. **Global State:** Context API para estado compartido

Context Providers:

```
/**
 * AuthContext: Estado de autenticación
 * ThemeContext: Tema de la aplicación
 * CartContext: Estado del carrito (cliente)
 * NotificationContext: Sistema de notificaciones
 */
```

Componentes UI

Sistema de Componentes:

- **Atomic Design:** Componentes organizados por complejidad
- **Radix UI:** Base para componentes complejos
- **Tailwind CSS:** Estilos utility-first
- **Variants:** Sistema de variantes con CVA

Estructura de Componentes:

```
components/
├── ui/                                # Componentes base (Radix + Tailwind)
│   ├── button.tsx                    # Componente Button
│   ├── input.tsx                     # Componente Input
│   ├── dialog.tsx                    # Componente Modal
│   └── ...
├── forms/                             # Componentes de formularios
├── layout/                             # Componentes de layout
├── charts/                             # Componentes de gráficos
└── domain/                             # Componentes específicos del dominio
```

Características de UI:

- **Responsive Design:** Adaptable a todos los dispositivos
- **Dark Mode:** Soporte para tema oscuro
- **Accessibility:** Componentes accesibles por defecto
- **Animation:** Animaciones suaves con Tailwind

Configuración de Build

Build Process:

1. **TypeScript Compilation:** Verificación de tipos
2. **Next.js Build:** Optimización y bundling
3. **Static Generation:** Pre-renderizado de páginas estáticas
4. **Image Optimization:** Optimización automática de imágenes

Optimizaciones:

- **Code Splitting:** División automática de código

- **Tree Shaking:** Eliminación de código no utilizado
- **Bundle Analysis:** Análisis de tamaño de bundles
- **Compression:** Compresión gzip/brotli

Shared - Módulo Compartido

Propósito y Función

El módulo **shared** centraliza tipos, validaciones y utilidades comunes entre frontend y backend, garantizando consistencia y reduciendo duplicación de código.

Beneficios:

- **Type Safety:** Tipos compartidos entre frontend y backend
- **DRY Principle:** Evita duplicación de código
- **Consistency:** Validaciones consistentes
- **Maintainability:** Cambios centralizados

Estructura del Módulo

```
shared/
├── common/           # Utilidades y tipos comunes
├── entities/         # Definiciones de entidades
│   ├── user/         # Tipos y validaciones de usuario
│   ├── product/      # Tipos y validaciones de producto
│   ├── order/        # Tipos y validaciones de orden
│   ├── promotion/    # Tipos y validaciones de promoción
│   └── category/     # Tipos y validaciones de categoría
├── enums/           # Enumeraciones del sistema
├── transactions/    # Tipos de transacciones
├── dist/            # Código compilado
├── index.ts         # Punto de entrada principal
├── package.json     # Configuración del paquete
└── tsconfig.json    # Configuración TypeScript
```

Tipos y Entidades

Entidades Principales:

```
// User Entity
interface IUser {
  _id: string;
  name: string;
  email: string;
  role: UserRole;
  isActive: boolean;
  createdAt: Date;
  updatedAt: Date;
}

// Product Entity
interface IProduct {
```

```

    _id: string;
    name: string;
    description: string;
    price: number;
    category: string;
    images: string[];
    isActive: boolean;
    nutritionalInfo?: NutritionalInfo;
}

// Order Entity
interface IOrder {
    _id: string;
    orderNumber: string;
    customer: CustomerInfo;
    items: OrderItem[];
    status: OrderStatus;
    total: number;
    createdAt: Date;
}

```

Enumeraciones:

```

enum UserRole {
    ADMIN = "ADMIN",
    MANAGER = "MANAGER",
    CHEF = "CHEF",
    SERVER = "SERVER",
    CASHIER = "CASHIER"
}

enum OrderStatus {
    PENDING = "PENDING",
    PREPARING = "PREPARING",
    READY = "READY",
    DELIVERED = "DELIVERED",
    CANCELLED = "CANCELLED"
}

enum PromotionType {
    DISCOUNT = "DISCOUNT",
    BOGO = "BOGO",
    BUNDLE = "BUNDLE",
    FREE_SHIPPING = "FREE_SHIPPING",
    GIFT_WITH_PURCHASE = "GIFT_WITH_PURCHASE",
    SEASONAL = "SEASONAL"
}

```

Validaciones con Zod

Esquemas de Validación:

```
// User Validation Schema
const UserCreateSchema = z.object({
  name: z.string().min(2).max(50),
  email: z.string().email(),
  password: z.string().min(8),
  role: z.nativeEnum(UserRole).optional()
});

// Product Validation Schema
const ProductCreateSchema = z.object({
  name: z.string().min(3).max(100),
  description: z.string().min(10),
  price: z.number().positive(),
  category: z.string(),
  images: z.array(z.string()).min(1)
});

// Order Validation Schema
const OrderCreateSchema = z.object({
  customer: CustomerInfoSchema,
  items: z.array(OrderItemSchema).min(1),
  paymentMethod: z.nativeEnum(PaymentMethod)
});
```

Beneficios de Zod:

- **Runtime Validation:** Validación en tiempo de ejecución
- **Type Inference:** Inferencia automática de tipos
- **Error Messages:** Mensajes de error descriptivos
- **Composability:** Esquemas reutilizables y componibles

Infraestructura y Despliegue

Configuración de Desarrollo

Requisitos del Sistema:

- **Node.js:** v20.x o superior
- **npm:** v10.x o superior
- **MongoDB:** v7.x o superior
- **Git:** Para control de versiones

Setup de Desarrollo:

```
# Clonar repositorio
git clone <repository-url>
cd PROYECTO
```

```
# Instalar dependencias
```

```
npm install

# Configurar variables de entorno
cp .env.example .env

# Iniciar servicios de desarrollo
npm run dev:all
```

Scripts de Desarrollo:

```
{
  "dev:frontend": "cd frontend && npm run dev",
  "dev:backend": "cd backend && npm run dev",
  "dev:shared": "cd shared && npm run build:watch",
  "dev:all": "concurrently \"npm run dev:shared\" \"npm run dev:backend\" \"npm run dev:frontend\""
}
```

Variables de Entorno

Backend (.env):

```
# Server Configuration
PORT=3001
NODE_ENV=development

# Database
MONGODB_URI=mongodb://localhost:27017/colori

# JWT
JWT_SECRET=your-super-secret-jwt-key
JWT_EXPIRES_IN=7d

# File Upload
UPLOAD_PATH=./uploads
MAX_FILE_SIZE=5242880

# CORS
FRONTEND_URL=http://localhost:3000
```

Frontend (.env.local):

```
# API Configuration
NEXT_PUBLIC_API_URL=http://localhost:3001/api

# App Configuration
NEXT_PUBLIC_APP_NAME=Colori
NEXT_PUBLIC_APP_VERSION=1.0.0
```

Scripts de Build

Backend Build:

```
{
  "build": "tsc",
```

```
"start": "node dist/server.js",  
"dev": "tsx watch src/server.ts"  
}
```

Frontend Build:

```
{  
  "build": "next build",  
  "start": "next start",  
  "dev": "next dev --turbo" }  
}
```

Shared Build:

```
{  
  "build": "tsc",  
  "build:watch": "tsc --watch"  
}
```

Consideraciones de Producción

Optimizaciones de Producción:

- **Minificación:** Código minificado y comprimido
- **Caching:** Estrategias de cache agresivas
- **CDN:** Distribución de assets estáticos
- **Monitoring:** Monitoreo de performance y errores

Configuración de Servidor:

- **Process Manager:** PM2 para gestión de procesos
- **Reverse Proxy:** Nginx para balanceo de carga
- **SSL/TLS:** Certificados HTTPS
- **Environment:** Variables de entorno seguras

Seguridad

Autenticación JWT

Implementación de JWT:

```
/**  
 * Generación de Token:  
 * - Payload con información del usuario  
 * - Firma con secret key  
 * - Tiempo de expiración configurable  
 */  
  
/**  
 * Validación de Token:  
 * - Verificación de firma
```



```
* - Validación de expiración
* - Extracción de payload
*/
```

Mejores Prácticas:

- **Secret Rotation:** Rotación periódica de claves
- **Short Expiration:** Tokens de corta duración
- **Refresh Tokens:** Renovación automática
- **Secure Storage:** Almacenamiento seguro en cliente

Validación de Datos

Validación en Múltiples Capas:

1. **Frontend:** Validación inmediata en formularios
2. **Backend:** Validación en middleware
3. **Database:** Validación en esquemas Mongoose

Sanitización:

- **Input Sanitization:** Limpieza de datos de entrada
- **SQL Injection Prevention:** Uso de ODM/ORM
- **XSS Prevention:** Escape de contenido HTML
- **CSRF Protection:** Tokens CSRF en formularios

CORS y Headers

Configuración CORS:

```
const corsOptions = {
  origin: process.env.FRONTEND_URL,
  credentials: true,
  optionsSuccessStatus: 200
};
```

Security Headers:

```
app.use((req, res, next) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'DENY');
  res.setHeader('X-XSS-Protection', '1; mode=block');
  next();
});
```

Manejo de Archivos

Validación de Archivos:

- **File Type:** Validación de tipos MIME

- **File Size:** Límites de tamaño
- **File Name:** Sanitización de nombres
- **Virus Scanning:** Escaneo de malware (producción)

Almacenamiento Seguro:

- **Path Traversal Prevention:** Prevención de ataques de directorio
- **Access Control:** Control de acceso a archivos
- **Encryption:** Encriptación de archivos sensibles

Performance y Optimización

Optimizaciones del Frontend

Next.js Optimizations:

- **Static Generation:** Pre-renderizado de páginas
- **Image Optimization:** Optimización automática de imágenes
- **Code Splitting:** División automática de código
- **Bundle Analysis:** Análisis de tamaño de bundles

React Optimizations:

- **Memoization:** React.memo para componentes
- **useMemo/useCallback:** Optimización de cálculos
- **Lazy Loading:** Carga perezosa de componentes
- **Virtual Scrolling:** Para listas grandes

Optimizaciones del Backend

Database Optimizations:

- **Indexing:** Índices optimizados para consultas
- **Aggregation:** Pipelines de agregación eficientes
- **Connection Pooling:** Pool de conexiones
- **Query Optimization:** Consultas optimizadas

API Optimizations:

- **Pagination:** Paginación de resultados
- **Field Selection:** Selección de campos específicos
- **Compression:** Compresión gzip/brotli
- **Rate Limiting:** Limitación de requests

Caching

Frontend Caching:

- **Browser Cache:** Cache del navegador
- **Service Worker:** Cache offline
- **React Query:** Cache de datos del servidor
- **Static Assets:** Cache de assets estáticos

Backend Caching:

- **Memory Cache:** Cache en memoria (Redis)
- **Database Query Cache:** Cache de consultas
- **API Response Cache:** Cache de respuestas
- **CDN Cache:** Cache de CDN

Monitoreo y Logging

Sistema de Logs

Configuración de Logging:

```
/**
 * Logger personalizado con diferentes niveles:
 * - ERROR: Errores críticos
 * - WARN: Advertencias
 * - INFO: Información general
 * - DEBUG: Información de depuración
 */
```

Tipos de Logs:

- **Request Logs:** Logs de requests HTTP
- **Error Logs:** Logs de errores y excepciones
- **Performance Logs:** Métricas de performance
- **Security Logs:** Eventos de seguridad

Manejo de Errores

Error Handling Strategy:

```
/**
 * Middleware de manejo de errores:
 * - Captura errores no manejados
 * - Formatea respuestas de error
 * - Registra errores en logs
 * - Notifica errores críticos
 */
```

Error Types:

- **Validation Errors:** Errores de validación
- **Authentication Errors:** Errores de autenticación
- **Authorization Errors:** Errores de autorización
- **Database Errors:** Errores de base de datos
- **Network Errors:** Errores de red

Métricas

Métricas del Sistema:

- **Response Time:** Tiempo de respuesta de API
- **Throughput:** Requests por segundo
- **Error Rate:** Tasa de errores
- **Memory Usage:** Uso de memoria
- **CPU Usage:** Uso de CPU

Business Metrics:

- **Active Users:** Usuarios activos
- **Orders per Hour:** Órdenes por hora
- **Revenue:** Ingresos en tiempo real
- **Popular Products:** Productos más vendidos

Testing

Estrategia de Testing

Pirámide de Testing:

1. **Unit Tests:** Tests unitarios (70%)
2. **Integration Tests:** Tests de integración (20%)
3. **E2E Tests:** Tests end-to-end (10%)

Testing Tools:

- **Jest:** Framework de testing
- **React Testing Library:** Testing de componentes React
- **Supertest:** Testing de APIs
- **Cypress:** Testing E2E

Testing del Backend

Unit Tests:

```
/**
 * Tests de servicios:
 * - Lógica de negocio
 * - Validaciones
 * - Transformaciones de datos
 */
```

```
/**
 * Tests de controladores:
 * - Manejo de requests
 * - Respuestas HTTP
 * - Manejo de errores
 */
```

Integration Tests:

```
/**
 * Tests de API:
 * - Endpoints completos
 * - Autenticación
 * - Base de datos
 */
```

Testing del Frontend

Component Tests:

```
/**
 * Tests de componentes:
 * - Renderizado
 * - Interacciones
 * - Props
 * - Estados
 */
```

Hook Tests:

```
/**
 * Tests de hooks personalizados:
 * - Estado
 * - Efectos
 * - Funciones
 */
```

Documentación de API

Endpoints Principales

Authentication Endpoints:

```
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/refresh
GET /api/auth/me
```

User Management Endpoints:

```
GET    /api/users
POST   /api/users
GET    /api/users/:id
PUT    /api/users/:id
DELETE /api/users/:id
```

Product Management Endpoints:

```
GET    /api/products
POST   /api/products
GET    /api/products/:id
PUT    /api/products/:id
DELETE /api/products/:id
POST   /api/products/:id/images
```

Order Management Endpoints:

```
GET    /api/orders
POST   /api/orders
GET    /api/orders/:id
PUT    /api/orders/:id/status
DELETE /api/orders/:id
```

Promotion Management Endpoints:

```
GET    /api/promotions
POST   /api/promotions
GET    /api/promotions/:id
PUT    /api/promotions/:id
DELETE /api/promotions/:id
```

Report Endpoints:

```
GET /api/reports/sales
GET /api/reports/products
GET /api/reports/users
GET /api/reports/export/excel
```

Esquemas de Datos

Request/Response Schemas:

```
// Login Request
interface LoginRequest {
  email: string;
  password: string;
}

// Login Response
interface LoginResponse {
  success: boolean;
  data: {
    user: IUser;
    token: string;
    expiresIn: string;
  };
}
```

```

    };
}

// Product Create Request
interface ProductCreateRequest {
    name: string;
    description: string;
    price: number;
    category: string;
    images: string[];
    nutritionalInfo?: NutritionalInfo;
}

```

Códigos de Respuesta

HTTP Status Codes:

- **200 OK:** Operación exitosa
- **201 Created:** Recurso creado exitosamente
- **400 Bad Request:** Datos de entrada inválidos
- **401 Unauthorized:** No autenticado
- **403 Forbidden:** No autorizado
- **404 Not Found:** Recurso no encontrado
- **409 Conflict:** Conflicto de datos
- **422 Unprocessable Entity:** Error de validación
- **500 Internal Server Error:** Error interno del servidor

Error Response Format:

```

interface ErrorResponse {
    success: false;
    error: string;
    message: string;
    details?: ValidationErrors;
    timestamp: string;
    path: string;
}

```